Ephemeral Sandbox: A LayerStack Workspace OS for Parallel Coding Agents

Provisional working title

[AUTHOR INFORMATION NEEDED]
[AFFILIATION INFORMATION NEEDED]

Draft manuscript scaffold

Abstract

Provisional design-first draft—not submission text. Agent systems can fan out intelligence, but conventional operating-system workspaces provide no agent-level protocol for fanning concurrent writes back into one durable project. Coding-agent teams and swarms inspect changing code, perform multi-file edits, run tests, and start services; sharing one mutable workspace permits live-state interference, while copying workspaces alone defers integration. We present Ephemeral Sandbox, a workspace OS organized around LayerStack: one durable project history, many private executable sessions, and controlled publication. A sessionless `exec_command` tool call receives an implicit isolated session over a leased LayerStack snapshot. When its command ledger drains, the runtime captures the private overlay, reconciles the complete delta against the current head, and either publishes one new layer or returns a structured rejection. Eligible text divergence can use a bounded three-way merge; structural, binary, oversized, or conflicting changes reject. Explicit sessions let multiple command and file tool calls share private state before publication or discard. Source and correctness tests establish these mechanisms, but not formal security, semantic merge correctness, process-state rollback, resource coordination, or improved multi-agent throughput. Whether Ephemeral Sandbox raises the workload-dependent useful-work concurrency ceiling remains an empirical question.

1 Introduction

Draft.

2 Goals, Non-goals, and Threat-model Boundary

Draft.

3 System Model and Invariants

Draft.

4 Workspace Execution

Draft.

5 Capture and Publication

Draft.

Stage	Time driver	Live or peak space driver
Session creation	$O(L)$ manifest cloning, path resolution, lower validation, and mount configuration in userspace	$O(L)$ metadata per lease/session and $O(R)$ across live leases; fresh upper/work directories do not eagerly copy the project
Capture	$O(U)$ metadata/path work plus per-directory sorting	$O(U)$ change metadata; regular-file payload remains referenced on disk
Plan/resolve	Changed-path routing and repeated layered fingerprint probes; eligible merge depends on line count and edit distance	Fingerprints and validations scale with $C+F$; merge retains file, line, edit, output, origin, and trace state
Commit	$O(C \log C)$ path aggregation, at least $O(B_p)$ byte hashing/copy/sync, and $O(L)$ manifest work under one writer	The private upper can coexist with staging/final publication data; each unsquashed publish adds a manifest layer
Squash/GC	Manifest/lease planning, layered directory traversal, then a brief serialized commit	Replacement metadata coexists with old layers; leases retain otherwise obsolete history until safe release

Table 1: Operational cost model derived from the v1 source. Logical bytes are not physical allocated bytes; the latter requires filesystem measurements.

6 Lifecycle and Recovery

Draft.

7 Implementation and Operational Interface

7.1 Source-Derived Operational Cost Model

Ephemeral Sandbox does not have one context-free time or space complexity. Each session passes through stages with different scaling variables. Let L be manifest depth, S the number of live sessions, U the number of private-upper entries, C the number of captured changes, F the number of validated source paths, B_p the logical regular-file bytes published, Q the number of concurrent publishers, and $R = \sum_{i=1}^{S} L_i$ the layer references retained across live leases. Table 1 records source-derived drivers; it is not a performance result.

The durable head transition is serialized by design. In v1, the exclusive writer section includes active-head resolution, eligible text reconciliation, payload hashing and copying, synchronization, promotion, and manifest replacement. Private execution can therefore remain parallel while publication wait grows with Q, B_p, F, or merge work. This is a mechanism-level saturation hypothesis, not evidence that publication is the limiting stage for any workload.

Text reconciliation is bounded to 8 MiB per input, but that byte gate is not an independent memory bound for the line diff. For total line count n, reached edit distance D, and $K = \min(n, 200{,}000)$, the current implementation retains an $O(KD)$ frontier trace and can approach an $O(K^2)$ trace envelope. The evaluation must stress line count and edit shape in addition to file bytes.

7.2 Operational Interface

Draft pending PW3. This subsection will describe the role-separated management, runtime, and read-only observability clients and the final tagged catalog-derived contract.

8 Evaluation

8.1 RQ3: Latency, Storage, and Resource Scaling

RQ3 tests the source-derived cost model rather than assuming that worker count alone explains scaling. The protocol independently varies worker/publisher count, manifest depth, live-session count and lease age,

private-upper entry count and fan-out, changed and validated paths, published logical bytes, conflict class, and eligible-merge bytes, lines, similarity, and edit distance. Storage cases include small edits to large files, many small files, and incompressible controls. All factors, warmups, repeats, and stopping rules must be fixed before the final source freeze.

For each stage, we report latency distributions, CPU, RSS/PSS, I/O, failures, and exact phase boundaries. Publication additionally reports writer-lock wait and hold time and decomposes planning, resolution/merge, hashing, copy, synchronization, and manifest transition. Storage reports logical, allocated, shared, and exclusive bytes for upper, work, staging, published layers, and lease-retained history. Squash reports plan, build, commit, and garbage collection separately. No cost-model term becomes a paper result until it is reproduced from frozen raw artifacts and analysis code.

The remaining RQ1–RQ5 methodology and all numerical results are pending protocol and evidence lock.

9 Limitations and Related Work

9.1 Cost and Implementation Limits

Workspace isolation removes one source of interference; it does not solve task decomposition, model quality, semantic compatibility, verification, resource ownership, or general agent coordination. v1 also exposes concrete scaling risks. It serializes publication and performs current-head reconciliation plus payload hashing, copying, and synchronization inside the writer critical section. It retains a full manifest per live lease, repeatedly probes layered history during path validation, and can retain obsolete history while long-lived leases protect session views. Its text merge is byte-limited but does not yet have an independently small line/edit-distance trace bound. These are source-derived limitations whose practical severity remains to be measured.

The copy-on-write workspace abstraction is not a claim of constant or negligible physical storage. Kernel copy-up, sparse allocation, compression, extent sharing, page cache, and filesystem metadata are backend-dependent. A small edit to a large lower file can produce a large logical upper file, and v1 publication copies the resulting regular-file payload into a new layer. Accordingly, the evaluation distinguishes logical from allocated, shared, and exclusive bytes.

9.2 LayerStack 2.0 as a Candidate Evolution

LayerStack 2.0 is future work, not part of the v1 contribution. Its migration protocol proposes a qualified single-filesystem storage domain and compares a copy-based v1 layout, a co-located copy control, and a reflink-required candidate. The target is to reduce byte-transfer and allocated-storage amplification without changing image compatibility, blame, squash, remount, active execution, crash, memory, daemon-health, or privilege behavior. A reflink is not $O(1)$: clone work depends on filesystem and extent structure, and later writes allocate changed extents.

The completed Windows feasibility evidence is negative and narrowly scoped. On the pinned stock Windows Docker Desktop/WSL 2 storage cell, direct `FICLONE` returned `errno=95`; the integrated Windows candidate therefore remained blocked. A future design requires capability detection and a correctness-preserving copy fallback, and this paper makes no Windows reflink storage or performance claim.

Other targets follow directly from the v1 model: shorten the writer critical section while preserving all-or-none and crash visibility semantics; share or compact immutable manifest state across leases; index revision-keyed path, fingerprint, and ignore lookups; and reject merge work through independently bounded CPU/memory admission.

9.3 Related Work

Draft pending PW4 and final citation verification.

10 Conclusion

Draft.

References